

MECHANISTIC EVALUATION OF TRANSFORMERS AND STATE SPACE MODELS

Aryaman Arora Neil Rathi Nikil Roashan Selvam Róbert Csordás
 Dan Jurafsky Christopher Potts
 Stanford University
 {aryamana, jurafsky, cgpotts}@stanford.edu

ABSTRACT

State space models (SSMs) for language modelling promise an efficient and performant alternative to quadratic-attention Transformers, yet show variable performance on recalling basic information from the context. While performance on synthetic tasks like Associative Recall (AR) can point to this deficiency, behavioural metrics provide little information as to *why*—on a mechanistic level—certain architectures fail and others succeed. To address this, we conduct experiments on AR, and find that only Transformers and Based SSM models fully succeed at AR, with Mamba and DeltaNet close behind, while the other SSMs (H3, Hyena) fail. We then use causal interventions to explain why. We find that Transformers and Based learn to store key–value associations in-context using induction. By contrast, the SSMs seem to compute these associations only at the last state using a single layer. We further investigate the mechanism underlying the success of Mamba, and find novel evidence that Mamba *does* implement induction: not via the SSM, but instead via short convolutions. Further experiments on a new hierarchical retrieval task, Associative Treecall (ATR), show that all architectures learn the same mechanism as they did for AR. Furthermore, we show that Mamba can learn Attention-like induction on ATR when short convolutions are removed. These results reveal that architectures with similar accuracy may still have substantive differences, motivating the adoption of mechanistic evaluations.

 github.com/aryamanarora/tinylang

1 INTRODUCTION

Transformers with quadratic attention remain the dominant architecture in language modelling despite numerous proposed efficient alternatives. Most notably, **state space models** (SSMs) achieve impressive perplexities and benchmark scores (e.g. Gu & Dao, 2024). Yet, SSMs exhibit deficiencies that benchmarks often fail to capture; for example, they struggle to perform **retrieval**, i.e. copying from the context (Jelassi et al., 2024; Wen et al., 2024; Waleffe et al., 2024; Bick et al., 2025).

Controlled synthetic tasks can make these limitations clear by isolating specific capabilities and enabling expressive experimentation at small scales across architectures. Particularly, much work has used the **associative recall** (AR) task as a testbed for studying in-context retrieval across architectures. In turn, AR has informed the design of novel LM architectures (e.g. Based; Arora et al., 2024b).

However, in this prior work, performance on synthetic tasks is measured solely via behavioural metrics like task accuracy. This is a missed opportunity: an advantage of these tasks is that they are designed to isolate a *specific behaviour* that implicates a mechanistic solution. For example, LMs should solve AR by storing *key–value* associations in-context at the *value*. For Transformer-based LMs (Vaswani et al., 2017), the mechanism is generally thought to be the **induction head**. (Olsson et al., 2022; Fu et al., 2023). We should therefore directly check whether each architecture learns an induction mechanism, as a way of checking that it has learned a robust solution to the task.

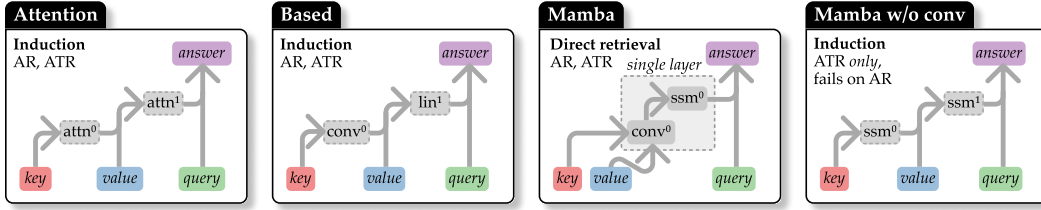


Figure 1: **Associative mechanisms in Attention, Based, and Mamba:** Key results from applying mechanistic metrics to understand how Attention and SSMs solve AR and our new retrieval task, Associative Treecall (ATR). Attention and Based both implement induction, but with different architectural components (§4.2 and §5.2). Mamba instead uses a single layer for both association and retrieval, but uses short convolution for association just like Based (§4.3 and §4.4). When convolutions are removed, Mamba implements induction, but only on ATR (§5.3).

Here, we propose using tools from mechanistic interpretability to directly analyse the mechanisms used to solve AR tasks. We use **causal interventions** (Geiger et al., 2024) on model internals to understand how these tasks are learned and implemented across a variety of architectures (§3). This allows us to track the emergence (or lack thereof) of the correct association and retrieval mechanisms inside the model, which moves us beyond observed task accuracy and towards a detailed picture of the solutions different architectures learn.

Through comprehensive experiments on AR, we find that all SSMs except Based do not use induction in the traditional sense to implement AR (§4.2); instead, we find that the best-performing SSM, Mamba, relies heavily on the short convolution component in each layer to perform *key-value* association, and fails to learn AR at all without this (§4.3). Through layer-internal interventions, we discover that the short convolution component implements an induction-like associative mechanism in Mamba, with the SSM component only being used for retrieval (§4.4).

To deepen our findings, we introduce **Associative Treecall** (ATR), a novel retrieval task more similar to real-world natural language retrieval than AR (§5). ATR uses a probabilistic context-free grammar (PCFG) to generate hierarchical data, on which we ask AR-like queries; the key difference from AR is that *keys* and their associated *values* need not be adjacent to each other. We find that the same mechanisms are implicated across architectures on ATR as on AR, confirming the generality of our findings (§5.2). However, unlike AR, Mamba does not need short convolutions to learn ATR; instead, mechanistic metrics reveal that Mamba falls back to an Attention-like two-layer induction mechanism when its short convolutions are removed (§5.3).

We offer a framework for better understanding and evaluating task performance via mechanistic interpretability. We demonstrate multiple examples of mechanistic evaluations revealing fundamental differences between architectures that are unknown via behavioural performance, summarised in Figure 1. We thus introduce mechanistic metrics as a new tool for architecture analysis.

2 RELATED WORK

Associative Recall. Our work relates to a variety of extant synthetic retrieval tasks. The foremost example, Associative Recall (AR),¹ is a synthetic task that evaluates in-context retrieval for language model architectures. AR has been used extensively, from early work on recurrent neural networks (Graves et al., 2014; Ba et al., 2016; Danihelka et al., 2016; Zhang & Zhou, 2017) to modern SSMs (Fu et al., 2023; Poli et al., 2023; Lutati et al., 2023; Jelassi et al., 2024; Arora et al., 2024a;b; Gu & Dao, 2024; Dao & Gu, 2024; Trockman et al., 2024; Liu et al., 2024a; Okpeke & Orvieto, 2025; Li et al., 2025b; Wang et al., 2025).

An AR task consists of a sequence of key-value pairs followed by a single *query* key; the goal is to produce the corresponding value for the given query. For example,

$$(1) \quad A \ 2 \ C \ 3 \ F \ 9 \ D \ 1 \ C \rightarrow 3$$

¹Also known as *associative retrieval*, *associative memory*, or *induction*.

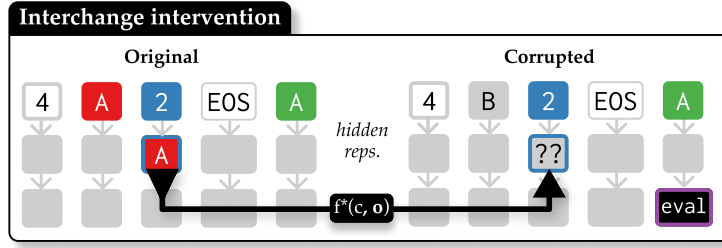


Figure 2: An example interchange intervention where we corrupt the *key* (A) and attempt to restore it by intervening at the *value* token in an intermediate representation. We evaluate the downstream effect on the next-token prediction at the *query* (which should ideally predict the *answer*).

Here, the correct next token is 3, since it is the value associated with the key C in context.

Despite being synthetic, AR has a direct analogue in natural language: *induction*, referring to in-context copying of sequences (Elhage et al., 2021; Olsson et al., 2022). Arora et al. (2024a;b) show that architecture-level improvements on AR translate directly to induction. Association is additionally widely studied in cognitive science as *binding*. Binding in neural networks has been examined by Greff et al. (2020); Kim & Schuster (2023); Feng & Steinhardt (2024); Prakash et al. (2024); Li et al. (2025a); Prakash et al. (2025).

Mechanistic interpretability. In order to measure the contribution of individual model components (neurons, layers, etc.) to output behaviour, we can apply causal interventions on neural network internals (Geiger et al., 2021; 2024). Informally, the core idea is to overwrite an activation at a specific component using a counterfactual input. If this changes model behaviour, then that component is causally relevant to the mechanism underlying that behaviour.

Some prior work in mechanistic interpretability has studied how some language models solve in-context retrieval tasks like induction and multiple choice question answering (Olsson et al., 2022; Lieberum et al., 2023; Brinkmann et al., 2024; Wiegrefe et al., 2025; Bick et al., 2025), as well as the training dynamics of Transformers on toy tasks using mechanistic metrics (Nanda et al., 2023; Reddy, 2024; Singh et al., 2024; Edelman et al., 2024; Tigges et al., 2024; Yin & Steinhardt, 2025). Yet thus far, *architectural comparisons* on synthetic tasks have not made use of causal interventions.

3 MECHANISTIC METRICS

Behavioural metrics provide little information as to *why* certain architectures succeed or fail on tasks of interest. Mechanistic metrics, which directly measure how information flows across model components and token positions, can tell us how AR and similar tasks are being solved by different architectures, and thus help us understand failures. We illustrate our approach in Figure 2.

We use interchange interventions (Geiger et al., 2021; 2024) to understand and measure how solutions to AR are implemented across architectures. We introduce this operation and define the resulting metrics for our tasks below. Our implementation uses the pyvene library (Wu et al., 2024).

Interchange intervention. Consider a language model $p(\cdot)$ and some input $\mathbf{b}$. We select a component f inside that model which computes some internal representation $f(\mathbf{b})$ during the LM’s forward pass. Now, consider a counterfactual input $\mathbf{s}$ which produces a counterfactual representation $f(\mathbf{s})$ when processed by f . We want to understand what about the output of p is dependent on f . Therefore, we perform an intervention which replaces the output $f(\mathbf{b})$ with that of $f(\mathbf{s})$ during the computation of $p(\mathbf{b})$, with the change propagating downstream. The result is notated $p_{f \leftarrow f^*}(\mathbf{b}, \mathbf{s})$.

Concrete setup for AR and ATR. We take $\mathbf{o}$ to be a ground-truth document from our data distribution and $\mathbf{c}$ to be a version of that document with exactly one important token corrupted: the *key* (see Figure 2). This corruption significantly reduces task accuracy by removing information that is necessary to answer the AR query.